```
var_dump($_SERVER);

Output:
array(67) {
  ["HOSTNAME"]=>
  string(12) "e117b73245f4"
  ["PHP_INI_DIR"]=>
  string(18) "/usr/local/etc/php"
  ...
}
```

These data structures are extremely important. They contain vital information about the environment PHP is running in as it executes your code. I stress again: even though it is possible to do so, do NOT write to any part of the `$GLOBALS` array. Sooner or later some other component (e.g., a third-party library) is going to rely on the information in there. Wrap those values up in your own functions and classes.

If you have even a basic understanding of how web servers work, then `$GLOBALS['_GET']`, `$GLOBALS['_POST']`, `$GLOBALS['_COOKIE']`, and so on should be self-explanatory (they hold HTTP request parameters, cookie data, etc.). Those of you that have ever worked with a shell script or similar should be able to guess at what `$GLOBALS['argv']` and `$GLOBALS['argc']` are for (storing information about how PHP was invoked at the command line). We'll see these superglobals again in relevant chapters.

Referencing

Did you notice in the previous section that the `$GLOBALS` array contained an element named `GLOBALS`, and the value was given as `*RECURSION*`? The reason this is displayed is because the `$GLOBALS` array contains a *reference*: a special variable that acts as an alias for another variable. In this case, the value of `$GLOBALS['GLOBALS']` is a reference for… `$GLOBALS`. In other words, it refers to itself, and `var_dump()` wisely decides not to output the same data again (and again, and again, and… you get the idea).

Think of references as one variable pointing to another. PHP will follow the chain until it gets to the "real" variable and work on that. This might sound complex but will probably make more sense if we see it in action: